

Machine Learning Model Development

We will cover:

1. **ML Pipelines**
 2. **Cross-Validation**
 3. **Hyperparameter Tuning**
 - GridSearchCV
 - RandomizedSearchCV
 - Optuna
 4. How they work together
 5. A complete practical example
-

1. Why Do We Need These Techniques?

Suppose you're building a model to predict whether a transaction is fraudulent.

Your workflow might look like:

```
Raw Data
  ↓
Clean Data
  ↓
Scale Features
  ↓
Train Model
  ↓
Tune Parameters
  ↓
Evaluate
```

But there are problems.

Problem 1 — Data leakage

What if you calculate your scaler using the entire dataset before splitting?

```
Entire Dataset
  ↓
StandardScaler
  ↓
Train/Test Split
```

Information from the test set has influenced the transformation.

That's **data leakage**.

Problem 2 — One train/test split may be lucky or unlucky

Suppose:

Split 1 → 91% accuracy

Maybe another split gives:

Split 2 → 84%

Which one should you trust?

This is where **cross-validation** helps.

Problem 3 — Models have settings

For example:

```
RandomForestClassifier(  
    n_estimators=100,  
    max_depth=10  
)
```

Why 100 trees?

Why depth 10?

These are **hyperparameters**.

We can search for better values.

That's where:

- Grid Search
- Random Search
- Optuna

come in.

2. What Is a Machine Learning Pipeline?

A pipeline is a sequence of machine-learning steps connected together.

Instead of manually doing:

```
scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)

model.fit(X_train_scaled, y_train)

X_test_scaled = scaler.transform(X_test)

predictions = model.predict(X_test_scaled)
```

we can create one pipeline.

```
Input Data
  ↓
Preprocessing
  ↓
Feature Transformation
  ↓
Model
  ↓
Prediction
```

3. Pipeline Analogy

Imagine a factory.

Raw materials enter:

```
Raw Material
  ↓
Cleaning
  ↓
Processing
  ↓
Assembly
  ↓
Quality Control
  ↓
Finished Product
```

An ML pipeline works similarly:

```
Raw Features
```

↓
Scaling
↓
Feature Selection
↓
Model
↓
Prediction

The advantage is that the entire process becomes a **single reproducible workflow**.

4. Simple Scikit-Learn Pipeline

Suppose we're using KNN.

KNN requires feature scaling.

We can write:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier())
])
```

Now:

```
pipeline.fit(X_train, y_train)
```

and:

```
predictions = pipeline.predict(X_test)
```

The pipeline automatically performs:

```
X_train
↓
StandardScaler
↓
KNN
```

5. Why Pipelines Are Important

5.1 They reduce data leakage

This is one of the most important reasons.

Suppose you're performing cross-validation.

Without a pipeline, someone might accidentally do:

```
scaler.fit_transform(X)
```

before cross-validation.

That means information from validation folds can influence preprocessing.

With a pipeline:

```
Fold 1
├─ preprocessing fitted on training portion
└─ model trained
```

```
Fold 2
├─ preprocessing fitted on training portion
└─ model trained
```

```
Fold 3
├─ preprocessing fitted on training portion
└─ model trained
```

Each fold keeps its validation data isolated.

6. Pipelines Make Experiments Reproducible

Instead of having several separate pieces of code:

```
scaler.fit(...)
transform(...)
feature_selection(...)
model.fit(...)
```

you have:

```
pipeline.fit(X_train, y_train)
```

This makes your workflow easier to:

- reproduce

- debug
 - deploy
 - tune
 - maintain
-

7. Pipeline With PCA

Remember PCA from the previous lesson.

Suppose we want:

```
Scaling
  ↓
PCA
  ↓
SVM
```

We can implement:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2)),
    ("model", SVC())
])

pipeline.fit(X_train, y_train)

predictions = pipeline.predict(X_test)
```

This is much cleaner than manually performing each step.

8. Cross-Validation

Now let's talk about **cross-validation**.

The problem with a single train/validation split is that the result can depend heavily on which observations happened to land in each split.

Cross-validation reduces this dependence.

9. K-Fold Cross-Validation

The most common approach is:

K-Fold Cross-Validation

Suppose:

$K = 5$

We divide our training data into five parts:

Fold 1
Fold 2
Fold 3
Fold 4
Fold 5

Then we train five times.

Round 1

Train: Fold 2 + 3 + 4 + 5
Validate: Fold 1

Round 2

Train: Fold 1 + 3 + 4 + 5
Validate: Fold 2

Round 3

Train: Fold 1 + 2 + 4 + 5
Validate: Fold 3

Round 4

Train: Fold 1 + 2 + 3 + 5
Validate: Fold 4

Round 5

Train: Fold 1 + 2 + 3 + 4
Validate: Fold 5

Then we calculate the average score.

10. Visual Representation

5-Fold Cross Validation

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
V	T	T	T	T
T	V	T	T	T
T	T	V	T	T
T	T	T	V	T
T	T	T	T	V

Where:

T = Training
V = Validation

Every observation gets an opportunity to be part of the validation set.

11. Why Not Use the Test Set for Cross-Validation?

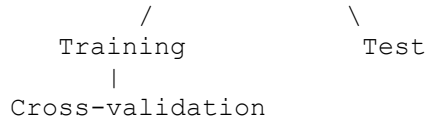
This is extremely important.

Suppose you have:

Dataset
↓
Training set
↓
Test set

Then perform cross-validation **inside the training set**.

Dataset
|
Train / Test Split
/ \



The test set should remain untouched until the final evaluation.

12. Python Cross-Validation

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    random_state=42
)

scores = cross_val_score(
    model,
    X_train,
    y_train,
    cv=5,
    scoring="accuracy"
)

print(scores)
print("Mean:", scores.mean())
```

You might get:

```
[0.91 0.88 0.90 0.92 0.89]
```

Mean:

```
0.90
```

So we can report approximately:

5-fold cross-validation accuracy = 90%

along with the fold-to-fold variation.

13. Stratified K-Fold

For classification, we often use:

StratifiedKFold

Why?

Suppose our dataset contains:

90% → Class A
10% → Class B

We want each fold to have approximately the same class proportions.

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

Then:

```
scores = cross_val_score(
    model,
    X_train,
    y_train,
    cv=cv,
    scoring="f1"
)
```

This is especially useful for imbalanced classification problems.

14. Cross-Validation for Regression

For regression, standard K-Fold is often appropriate:

```
from sklearn.model_selection import KFold

cv = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

Then:

```
scores = cross_val_score(
    model,
    X_train,
```

```
y_train,  
cv=cv,  
scoring="neg_mean_absolute_error"  
)
```

Note that scikit-learn represents losses such as MAE as negative scores when using its "higher is better" scoring convention.

15. Cross-Validation and Model Selection

Suppose we compare:

```
Model A → 85%  
Model B → 89%  
Model C → 91%
```

But these are based on one split.

Instead, cross-validation might show:

Model	CV Mean Accuracy
Logistic Regression	84%
Decision Tree	87%
Random Forest	91%
SVM	90%

This gives us a more robust basis for comparing models on the training data.

16. What Is Hyperparameter Tuning?

A model has:

Parameters

Learned from data.

For a linear model:

$$y=wx+by=wx+b$$

The model learns:

w
 b

These are **parameters**.

Hyperparameters

Settings we choose before/during model training.

For Random Forest:

```
n_estimators=100  
max_depth=10  
min_samples_leaf=2
```

These are hyperparameters.

The model doesn't normally learn these in the same way it learns its parameters.

17. Example: Random Forest Hyperparameters

Suppose:

```
RandomForestClassifier(  
    n_estimators=100,  
    max_depth=10  
)
```

We might ask:

Would 50 trees work better?

What about 200?

Should `max_depth` be 5?

What about 20?

We can search systematically.

18. Grid Search

Grid Search means:

Try every combination from the values we specify.

Suppose:

```
param_grid = {  
    "n_estimators": [50, 100, 200],  
    "max_depth": [5, 10, 20]  
}
```

Number of combinations:

$3 \times 3 = 9$

Grid Search evaluates all nine combinations.

19. GridSearchCV

```
from sklearn.model_selection import GridSearchCV  
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(  
    random_state=42  
)
```

```
param_grid = {  
    "n_estimators": [50, 100, 200],  
    "max_depth": [5, 10, 20],  
    "min_samples_leaf": [1, 2, 4]  
}
```

```
grid_search = GridSearchCV(  
    estimator=model,  
    param_grid=param_grid,  
    cv=5,  
    scoring="f1_weighted",  
    n_jobs=-1  
)
```

```
grid_search.fit(X_train, y_train)
```

Then:

```
print(grid_search.best_params_)
```

and:

```
print(grid_search.best_score_)
```

20. What Is GridSearchCV Actually Doing?

This is important.

It isn't simply:

```
Parameter combination
```

↓

```
Train once
```

↓

```
Score
```

Instead, with:

```
cv=5
```

each parameter combination is evaluated using 5-fold cross-validation.

For example:

```
Combination 1
```

↓

```
Fold 1
```

```
Fold 2
```

```
Fold 3
```

```
Fold 4
```

```
Fold 5
```

↓

```
Average score
```

```
Combination 2
```

↓

```
Fold 1
```

```
Fold 2
```

```
Fold 3
```

```
Fold 4
```

```
Fold 5
```

↓

```
Average score
```

```
...
```

This can become computationally expensive.

21. Grid Search Problem

Suppose you have:

```
n_estimators = [50, 100, 200, 400]
max_depth = [5, 10, 20, 30]
min_samples_split = [2, 5, 10]
min_samples_leaf = [1, 2, 4]
```

That's:

$4 \times 4 \times 3 \times 4 \times 3 \times 3 = 144 = 144$

parameter combinations.

With:

5-fold CV

you train:

$144 \times 5 = 720$

models.

And that's before considering additional preprocessing or other computational costs.

22. Randomized Search

Random Search solves this problem differently.

Instead of testing **every combination**, it randomly samples combinations from the search space.

```
from sklearn.model_selection import RandomizedSearchCV

param_distributions = {
    "n_estimators": [50, 100, 200, 400],
    "max_depth": [5, 10, 20, 30],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4]
}

random_search = RandomizedSearchCV(
```

```
estimator=model,  
param_distributions=param_distributions,  
n_iter=20,  
cv=5,  
scoring="f1_weighted",  
random_state=42,  
n_jobs=-1  
)  
  
random_search.fit(X_train, y_train)
```

Now instead of testing all 144 combinations, we're testing:

20 combinations

23. Grid Search vs Random Search

Grid Search	Random Search
Tests every specified combination	Tests randomly selected combinations
Can be expensive	Usually cheaper
Good for small search spaces	Good for larger search spaces
Exhaustive over specified grid	Doesn't guarantee testing every combination

24. Why Random Search Can Be Surprisingly Effective

Imagine you have:

10 hyperparameters

but only 2 or 3 have a major impact on performance.

Grid Search wastes resources exploring every combination.

Random Search can sometimes discover good values for the important parameters without exhaustively exploring the entire space.

This is why random search can be much more efficient in large search spaces.

25. Optuna

Now we get to a more advanced approach.

Optuna is a hyperparameter optimization framework.

Instead of manually specifying every combination, Optuna intelligently searches the parameter space.

The key idea is:

Use previous trial results to decide what to try next.

26. Optuna Analogy

Imagine you're trying to find the best temperature for baking bread.

You try:

```
180°C → good
200°C → excellent
220°C → burnt
```

Instead of randomly continuing forever, you use what you've learned to focus around the promising region.

Optuna works similarly.

27. Optuna Workflow

```
Define objective
  ↓
Suggest hyperparameters
  ↓
Train model
  ↓
Evaluate
  ↓
Return score
  ↓
Optuna learns from trial
  ↓
```

Suggest better region

↓
Repeat

28. Basic Optuna Example

First install:

```
pip install optuna
```

Then:

```
import optuna

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):

    n_estimators = trial.suggest_int(
        "n_estimators",
        50,
        300
    )

    max_depth = trial.suggest_int(
        "max_depth",
        3,
        30
    )

    min_samples_split = trial.suggest_int(
        "min_samples_split",
        2,
        20
    )

    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        random_state=42
    )

    scores = cross_val_score(
        model,
        X_train,
        y_train,
        cv=5,
        scoring="f1_weighted"
    )
```

```
return scores.mean()
```

Then:

```
study = optuna.create_study(  
    direction="maximize"  
)  
  
study.optimize(  
    objective,  
    n_trials=50  
)
```

Finally:

```
print(study.best_params)  
print(study.best_value)
```

29. What Makes Optuna Different?

Grid Search:

Try everything

Random Search:

Try random combinations

Optuna:

Use previous results to guide future trials

This makes it useful when:

- training is expensive
 - search spaces are large
 - parameters interact
 - you want efficient optimization
-

30. Hyperparameter Types in Optuna

Optuna supports different types of search spaces.

Integer

```
trial.suggest_int(  
    "max_depth",  
    3,  
    30  
)
```

Float

```
trial.suggest_float(  
    "learning_rate",  
    0.001,  
    0.1,  
    log=True  
)
```

Categorical

```
trial.suggest_categorical(  
    "kernel",  
    ["linear", "rbf", "poly"]  
)
```

31. Example: Tuning SVM with Optuna

```
import optuna  
  
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import StandardScaler  
from sklearn.svm import SVC  
from sklearn.model_selection import cross_val_score  
  
def objective(trial):  
  
    C = trial.suggest_float(  
        "C",  
        1e-3,  
        100,  
        log=True  
    )  
  
    gamma = trial.suggest_float(  
        "gamma",  
        1e-4,  
        1,  
        log=True  
    )  
  
    model = Pipeline([  
        ("scaler", StandardScaler()),  
        ("svm", SVC(  
            C=C,  
            gamma=gamma,  

```

```

        kernel="rbf"
    ))
])

scores = cross_val_score(
    model,
    X_train,
    y_train,
    cv=5,
    scoring="accuracy"
)

return scores.mean()

```

Then:

```

study = optuna.create_study(
    direction="maximize"
)

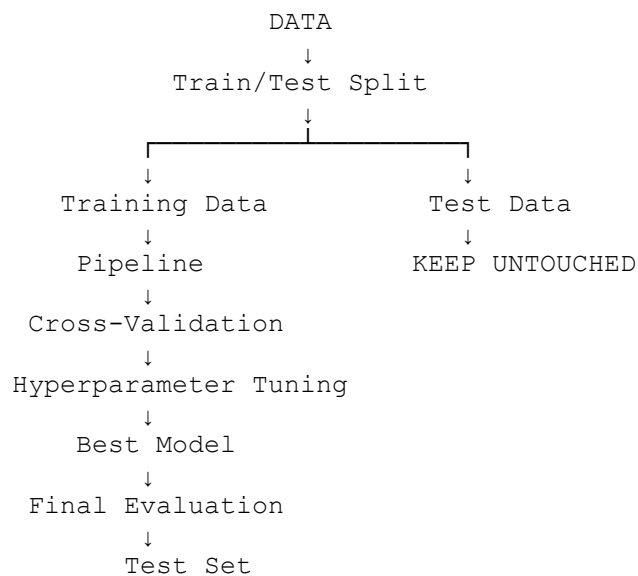
study.optimize(
    objective,
    n_trials=50
)

```

32. The Most Important Relationship

These concepts should not be thought about separately.

They work together:



This is a **very important workflow**.

33. Complete Example

Let's combine everything.

Suppose we have a classification problem.

We'll use:

```
StandardScaler
      ↓
SVM
```

and tune:

```
C
gamma
```

using:

```
Pipeline
+
5-fold CV
+
GridSearchCV
```

Step 1 — Import

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.datasets import load_breast_cancer
```

Step 2 — Load data

```
data = load_breast_cancer()

X = data.data
y = data.target
```

Step 3 — Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)
```

Notice:

We don't touch the test set during tuning.

Step 4 — Build pipeline

```
pipeline = Pipeline([  
    ("scaler", StandardScaler()),  
    ("svm", SVC())  
])
```

Step 5 — Define search space

```
param_grid = {  
    "svm__C": [0.1, 1, 10, 100],  
    "svm__gamma": ["scale", 0.01, 0.1, 1]  
}
```

Notice:

svm__C

The `svm__` tells scikit-learn that the parameter belongs to the pipeline step named:

svm

Step 6 — Grid Search

```
grid_search = GridSearchCV(  
    pipeline,  
    param_grid,  
    cv=5,  
    scoring="f1",  
    n_jobs=-1  
)  
  
grid_search.fit(X_train, y_train)
```

Step 7 — Find best parameters

```
print(grid_search.best_params_)
```

and:

```
print(grid_search.best_score_)
```

Step 8 — Evaluate on test data

```
from sklearn.metrics import classification_report

predictions = grid_search.predict(X_test)

print(
    classification_report(
        y_test,
        predictions
    )
)
```

This is a proper ML workflow.

34. Why We Don't Tune on the Test Set

This is one of the most important concepts to teach beginners.

Imagine:

```
Test accuracy = 90%
```

Then you change the hyperparameters because of that test result.

Then:

```
Test accuracy = 93%
```

Then you change them again.

You've effectively started **using the test set to make modeling decisions**.

The test set is no longer a truly independent final evaluation.

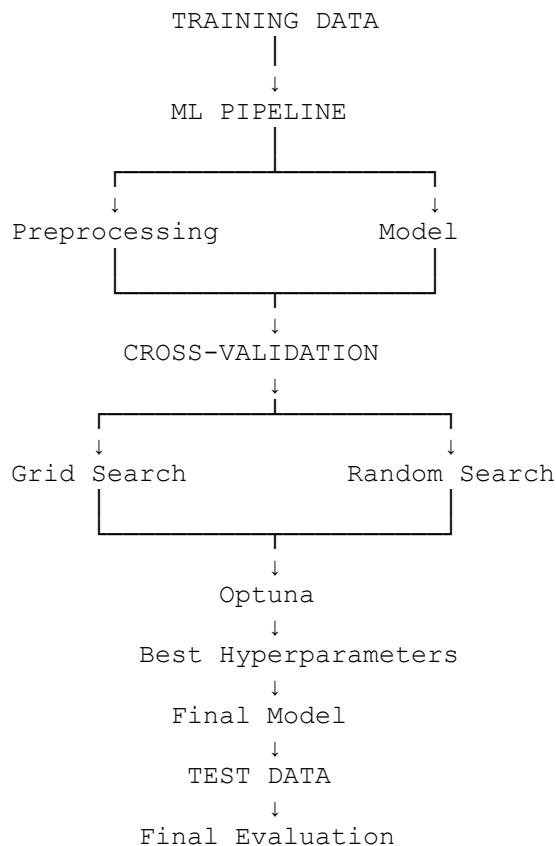
Instead:

```
Training set
↓
```


Cross-validation
↓
Hyperparameter tuning
↓
Select final model
↓
ONE final evaluation
↓
Test set

35. Pipeline + Cross-Validation + Hyperparameter Tuning

The complete relationship can be visualized as:



36. GridSearch vs RandomSearch vs Optuna

Feature	GridSearchCV	RandomizedSearchCV	Optuna
Search strategy	Exhaustive grid	Random sampling	Adaptive optimization

Feature	GridSearchCV	RandomizedSearchCV	Optuna
Easy to use	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★
Large search spaces	Poorer	Good	Excellent
Computational efficiency	Lower	Better	Often better
Uses previous trials	No	No	Yes
Good for beginners	Yes	Yes	After fundamentals
Complex optimization	Limited	Moderate	Strong

The stars here are **not performance rankings**; they are just rough teaching indicators of relative ease/fit for the stated use case.

37. Important Concepts to Remember

Pipeline

Connect preprocessing and modeling into one reproducible workflow.

Cross-validation

Evaluate a model across multiple training/validation splits.

Grid Search

Try every combination you specify.

Random Search

Try a selected number of randomly sampled combinations.

Optuna

Use an optimization strategy that learns from previous trials to search efficiently.

38. A Good Real-World Workflow

For your ML projects, especially projects like **fraud detection, agricultural disease classification, or medical AI**, a strong workflow is:

1. Collect data
- ↓
2. Clean data
- ↓
3. Explore data
- ↓
4. Train/Test Split
- ↓
5. Build Pipeline
- ↓
6. Select baseline model
- ↓
7. Cross-validation
- ↓
8. Hyperparameter tuning
- ↓
9. Select final model
- ↓
10. Evaluate ONCE on test set
- ↓
11. Error analysis
- ↓
12. Save model + preprocessing
- ↓
13. Deploy
- ↓
14. Monitor performance

One particularly important lesson

Do not let the test set become part of your experimentation loop.

The test set is supposed to answer:

"After all our decisions have been made, how does the final model perform on data it has not been used to make decisions about?"

That distinction is fundamental to trustworthy machine learning.

1. Imbalanced Data

What is imbalanced data?

A dataset is **imbalanced** when some classes have significantly more observations than others.

For example, suppose we are building a fraud detection model:

Transaction	Class
1	Legitimate
2	Legitimate
3	Legitimate
4	Legitimate
5	Legitimate
6	Fraud

We have:

- Legitimate: 5,000,000 transactions
- Fraud: 10,000 transactions

Fraud represents only about **0.2%** of the data.

This is an imbalanced dataset.

Real-world examples

Problem	Majority	Minority
Fraud detection	Legitimate	Fraud
Medical diagnosis	Healthy	Disease
Spam detection	Normal email	Spam
Manufacturing	Normal product	Defective
Network security	Normal traffic	Attack
Agriculture	Healthy plant	Diseased plant

This is particularly important in your **fraud detection and medical AI examples**, because the minority class can be the class we care most about.

2. Why Accuracy Can Be Misleading

Imagine:

99 people don't have a disease, and 1 person has the disease.

A model predicts:

"Nobody has the disease."

It gets:

99/100 = 99% accuracy

Sounds excellent.

But the model completely failed to identify the sick patient.

This is why we shouldn't automatically use accuracy for imbalanced classification.

Instead, we look at:

- Precision
 - Recall
 - F1-score
 - Specificity
 - ROC-AUC
 - PR-AUC
 - Confusion matrix
-

3. Confusion Matrix

For binary classification:

	Actual	
	Positive	Negative
Pred Positive	TP	FP
Pred Negative	FN	TN

TP — True Positive

Model predicted positive and it actually was positive.

TN — True Negative

Model predicted negative and it actually was negative.

FP — False Positive

Model predicted positive but it was actually negative.

FN — False Negative

Model predicted negative but it was actually positive.

4. Precision vs Recall

Precision

"When my model says positive, how often is it correct?"

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Example:

A fraud model flags 100 transactions as fraud.

Only 80 are actually fraud.

$$\text{Precision} = \frac{80}{100} = 80\%$$

Recall

"Of all the actual positive cases, how many did my model find?"

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Suppose there are actually 100 fraudulent transactions.

The model detects 80.

$$\text{Recall} = \frac{80}{100} = 80\%$$

Important distinction

For **disease detection**, recall can be extremely important.

Missing a sick patient can be more serious than sending a healthy patient for additional testing.

For **fraud detection**, you may care about both recall and precision because too many false alarms can inconvenience legitimate customers.

5. Techniques for Handling Imbalanced Data

There isn't one universal solution.

Method 1 — Undersampling

Reduce the majority class.

Before:

```
Majority = 10,000  
Minority = 1,000
```

After:

```
Majority = 2,000  
Minority = 1,000
```

Advantage

Faster training.

Disadvantage

You throw away data.

6. Oversampling

Increase the minority class.

For example:

Before:

```
Majority = 10,000
```

```
Minority = 1,000
```

After:

```
Majority = 10,000
```

```
Minority = 10,000
```

One simple method is:

Random oversampling

Duplicate minority examples.

But duplicated observations can cause overfitting.

7. SMOTE

A more sophisticated technique is **SMOTE**:

Synthetic Minority Oversampling Technique.

Instead of simply copying minority observations, SMOTE generates synthetic observations between existing minority samples.

Conceptually:

```
Minority A ----- Minority B
      \               /
       Synthetic point
```

Python:

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)

X_train_resampled, y_train_resampled = smote.fit_resample(
    X_train,
    y_train
)
```

VERY IMPORTANT

Never apply SMOTE before splitting the data.

Wrong:


```
Dataset
↓
SMOTE
↓
Train/Test Split
```

Correct:

```
Dataset
↓
Train/Test Split
↓
SMOTE → training data only
↓
Model
↓
Test data
```

Otherwise, information from synthetic training examples can leak into the test evaluation.

8. Class Weights

Another option is to tell the algorithm:

"Mistakes on the minority class should cost more."

For example:

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    class_weight="balanced",
    random_state=42
)
```

Similarly:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    class_weight="balanced"
)
```

This is often a good first approach before trying more complicated resampling techniques.

9. Feature Encoding

Now let's move to another common problem.

Machine learning algorithms generally expect numerical input.

But real-world data often contains categorical values.

For example:

Age	Gender	City	Purchased
-----	--------	------	-----------

23	Male	Lagos	Yes
----	------	-------	-----

35	Female	Ibadan	No
----	--------	--------	----

29	Male	Abuja	Yes
----	------	-------	-----

A model can't directly perform mathematical operations on:

Male
Female
Lagos
Ibadan

So we need **encoding**.

10. Label Encoding

Suppose:

Gender

Male
Female
Female
Male

We could convert:

Male → 0
Female → 1

Using:

```
from sklearn.preprocessing import LabelEncoder
```

```
encoder = LabelEncoder()

df["Gender"] = encoder.fit_transform(df["Gender"])
```

However, be careful.

For **nominal categories** such as:

```
Lagos
Ibadan
Abuja
Kano
```

using:

```
Lagos → 0
Ibadan → 1
Abuja → 2
Kano → 3
```

may incorrectly suggest that Kano > Abuja > Ibadan > Lagos.

There isn't necessarily such an ordering.

11. One-Hot Encoding

For nominal categories, one-hot encoding is often more appropriate.

Original:

```
City
----
Lagos
Ibadan
Abuja
```

Becomes:

Lagos	Ibadan	Abuja
1	0	0
0	1	0
0	0	1

Python:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    handle_unknown="ignore",
    sparse_output=False
)
```

The `handle_unknown="ignore"` part is particularly useful when deploying models because new categories may appear in production.

12. Ordinal Encoding

Sometimes categories actually have an order.

For example:

Education:

High School
Bachelor
Master
PhD

Here ordering makes sense.

We could represent:

High School → 0
Bachelor → 1
Master → 2
PhD → 3

This is called **ordinal encoding**.

13. Feature Scaling

Now imagine we have:

Age	Salary
25	50,000
30	70,000
40	150,000

The features have very different numerical scales.

Age → 20-80
Salary → 20,000-500,000

Some algorithms are sensitive to this.

Examples include:

- KNN
- SVM
- Logistic Regression
- Linear Regression
- Neural networks
- K-Means
- PCA

14. Standardization

Standardization transforms a feature approximately to:

- mean = 0
- standard deviation = 1

Formula:

$$z = \frac{x - \mu}{\sigma}$$

Python:

```
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
  
X_test_scaled = scaler.transform(X_test)
```

Notice:

```
fit_transform(X_train)
```

but:

```
transform(X_test)
```

We do not fit the scaler on the test set.

15. Min-Max Scaling

Another approach is Min-Max scaling.

It usually transforms values into:

$$0 \leq x' \leq 1$$

Formula:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Python:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

16. Which Scaling Method?

Method	Main idea
StandardScaler	Mean 0, std 1
MinMaxScaler	Usually 0–1
RobustScaler	Less sensitive to outliers

For example:

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
```

RobustScaler can be useful when your dataset contains substantial outliers.

17. Encoding + Scaling Together

Real datasets usually contain **both numerical and categorical features**.

Example:

Age	→ numerical
Salary	→ numerical
Gender	→ categorical
City	→ categorical

This is where `ColumnTransformer` becomes extremely useful.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_features = ["Age", "Salary"]
categorical_features = ["Gender", "City"]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)
```

Then combine it with a model:

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LogisticRegression())
])
```

Then:

```
pipeline.fit(X_train, y_train)

predictions = pipeline.predict(X_test)
```

This is a **real-world ML pattern**.

18. Why Pipeline Matters

Without a pipeline, students often end up doing:

```
Clean data
↓
```

Encode
↓
Scale
↓
Split
↓
Train

and accidentally leak information.

With a pipeline:

Raw data → Split {
 Scaling
 Encoding → Model
 Other preprocessing

The preprocessing is fitted appropriately within the training process.

This becomes particularly important when we introduce:

Cross-validation + hyperparameter tuning

because the preprocessing must happen separately inside each CV training fold.

19. Kaggle Practice

Now let's turn all of this into practical work.

I recommend teaching students through a **Kaggle workflow**, rather than only explaining concepts.

Kaggle Project 1 — Titanic

The classic beginner project:

Titanic: Machine Learning from Disaster

Students predict whether a passenger survived.

Typical features include:

PassengerId
Pclass

Name
Sex
Age
SibSp
Parch
Ticket
Fare
Cabin
Embarked

Target:

Survived

This dataset is excellent for teaching:

- Missing values
- Categorical encoding
- Numerical features
- Feature engineering
- Scaling
- Classification
- Train/test split
- Cross-validation
- Pipelines
- Model evaluation
- Kaggle submission

20. Titanic Practice Workflow

Step 1 — Load data

```
import pandas as pd

train = pd.read_csv("train.csv")

train.head()
```

Step 2 — Explore

```
train.shape
train.info()
train.describe()
train.isnull().sum()
```

Step 3 — Identify target

```
X = train.drop("Survived", axis=1)
y = train["Survived"]
```

Step 4 — Train/test split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

`stratify=y` is useful for classification because it helps preserve class proportions.

21. Handle Numerical Features

```
numeric_features = [
    "Age",
    "SibSp",
    "Parch",
    "Fare"
]
```

Categorical:

```
categorical_features = [
    "Sex",
    "Embarked",
    "Pclass"
]
```

Then:

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
```

Numerical pipeline:

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
```

```
        ("scaler", StandardScaler())
    ])
```

Categorical pipeline:

```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

22. Combine Everything

```
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])
```

Then model:

```
from sklearn.ensemble import RandomForestClassifier

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        random_state=42
    ))
])
```

Train:

```
model.fit(X_train, y_train)
```

Predict:

```
y_pred = model.predict(X_test)
```

Evaluate:

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

23. Kaggle Project 2 — Credit Card Fraud Detection

This is an excellent **intermediate project** for teaching imbalanced data.

The goal:

Transaction → Fraud / Not Fraud

The students should investigate:

Class distribution



Train/Test Split



Baseline model



Confusion Matrix



Precision

Recall

F1

ROC-AUC

PR-AUC



Class weights



SMOTE



Compare results

The key lesson should be:

Don't simply ask "What is my accuracy?" Ask "How does my model behave on the minority class?"

24. Kaggle Project 3 — House Prices

The Kaggle **House Prices** competition is excellent for regression.

Students learn:

Missing values



Categorical encoding



Numerical preprocessing



Feature engineering



Scaling



Regression
↓
Cross-validation
↓
Hyperparameter tuning

Evaluation can introduce:

- MAE
 - MSE
 - RMSE
 - R^2
 - Kaggle's competition-specific metric
-

25. A Good Kaggle Learning Path

For your mentorship class, I'd structure the practice like this:

Beginner

Titanic

Learn:

EDA
↓
Missing values
↓
Encoding
↓
Scaling
↓
Pipeline
↓
Classification
↓
Evaluation

Intermediate

House Prices

Learn:

Regression
↓
Feature engineering

↓
Encoding
↓
Scaling
↓
Cross-validation
↓
Hyperparameter tuning

Intermediate/Advanced

Credit Card Fraud Detection

Learn:

Imbalanced data
↓
Class weights
↓
SMOTE
↓
Precision/Recall
↓
F1
↓
ROC-AUC
↓
PR-AUC

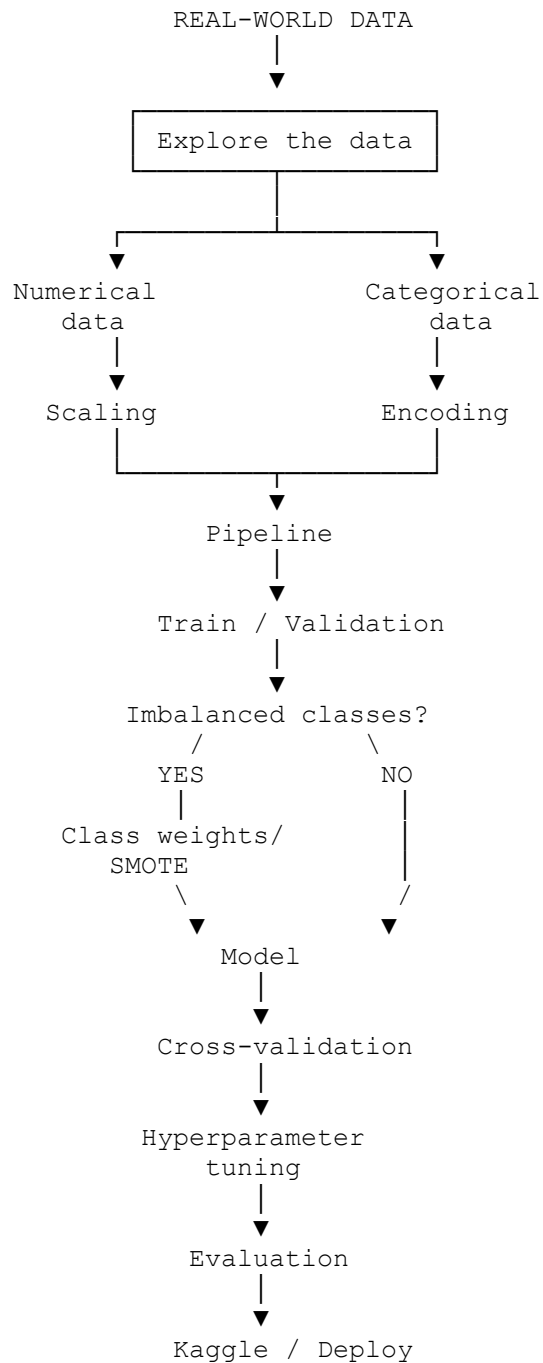
Advanced

Move into:

XGBoost / LightGBM
↓
Feature engineering
↓
Optuna
↓
Ensembling
↓
Kaggle competition
↓
Model interpretation
↓
Deployment

The Big Picture

These topics are actually connected:



The main lesson I would give your students

Machine learning is not just choosing an algorithm.

A large part of real ML work is:

getting the data into a form where the algorithm can learn correctly, without leaking information or being fooled by the dataset.

That is why **encoding, scaling, imbalance handling, pipelines, cross-validation, and tuning** are just as important as knowing KNN, Random Forest, SVM, etc.